7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **5**
1.1 Obiettivo del documento 5
1.2 Glossario 5
1.3 Riferimenti 5
1.3.1 Riferimenti normativi 5
1.3.2 Riferimenti informativi 5

2 Tecnologie utilizzate **7**

3 Linguaggio di Programmazione: TypeScript **7**
3.1 Deployment: Electron 7
3.2 Framework Frontend: Angular 8
3.3 Librerie Aggiuntive 8
3.4 Tecnologie di analisi statica 9
3.5 Tecnologie di analisi dinamica 9

4 Contesto del sistema **9**
4.1 Modello architetturale 10
4.2 Descrizione degli attori esterni 10

5 Architettura dei container **11**
5.1 Renderer 12
5.1.1 Struttura architetturale 12
5.2 [Main] 12
5.2.1 Tecnologia: Node.js 12

6 Architettura delle componenti **13**
6.1 [Componente 1] 13
6.1.1 Descrizione 13
6.1.2 Interfacce 13
6.1.3 Dipendenze 13
6.1.4 Tracciamento dei requisiti 13
6.2 [Componente 2] 13
6.2.1 Descrizione 13
6.2.2 Interfacce 13
6.2.3 Dipendenze 13
6.2.4 Tracciamento dei requisiti 13
6.3 Componenti di Ricerca (FrontEnd) 14
6.3.1 Descrizione 14

7 Backend - Indexing **14**
7.1 Panoramica 14
7.2 Componenti 14
7.2.1 ApplicationBootstrap, IndexDipUC 17

7.2.2 EmbeddingService, WordEmbeddingPort 19
7.2.3 LocalPackageReaderAdapter, DataMapper, DipParser, FileSystem-Provider 21

Elenco delle figure

1 Diagramma C1 - Contesto del sistema *DIPReader* 10
2 Diagramma C2 - Container del sistema *DIPReader* 11
3 Diagramma C3 - Componenti del modulo di indicizzazione 15
4 Diagramma C3 - Componenti del modulo di indicizzazione 16
5 Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione 17
6 Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione 19
7 Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione 21
8 Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione 22
9 Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione 23
10 Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione 24

Elenco delle tabelle

2 Dettagli sulla scelta di TypeScript come linguaggio di programmazione . 7
4 Dettagli sulla scelta di Electron come piattaforma di deployment 7
6 Dettagli sulla scelta di Angular come framework frontend 8
8 Dettagli sulle scelte di librerie aggiuntive 8
10 Dettagli sulle scelte di tecnologie di analisi statica 9
12 Dettagli sulle scelte di tecnologie di analisi dinamica 9

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Tabella 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Tabella 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Tabella 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Tabella 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Tabella 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Tabella 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

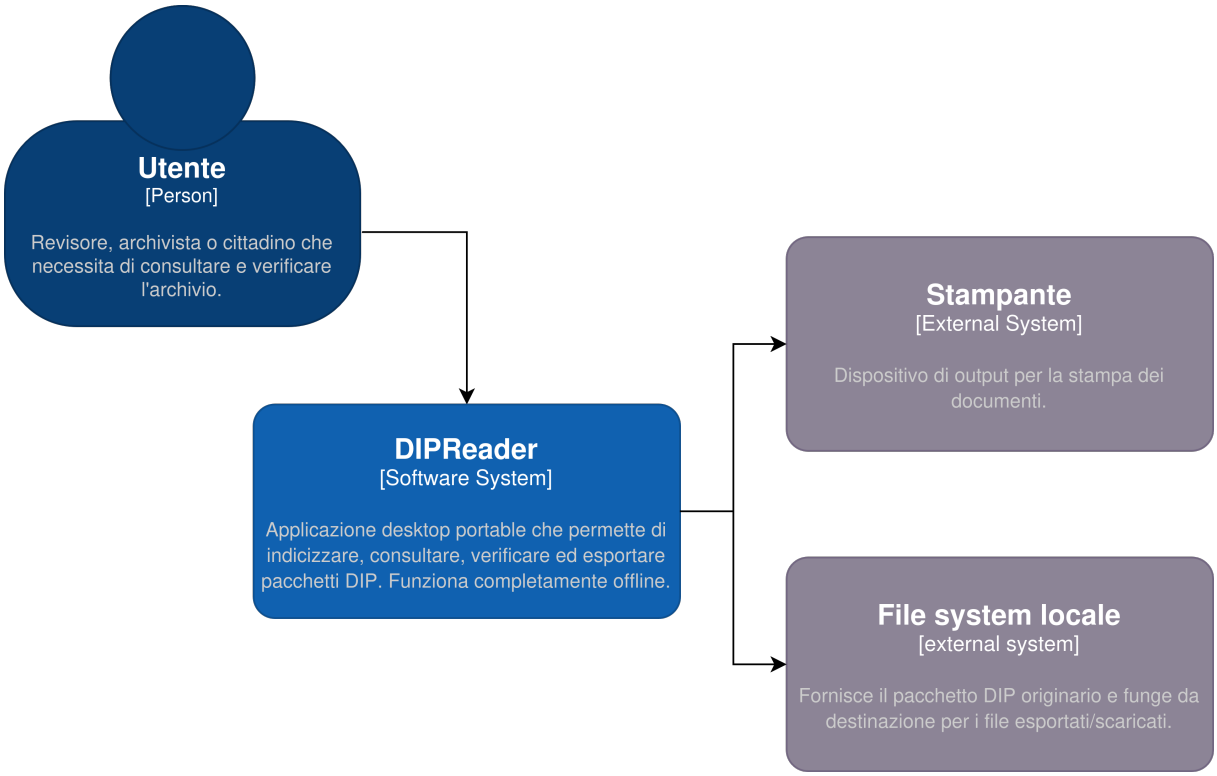

Figura 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

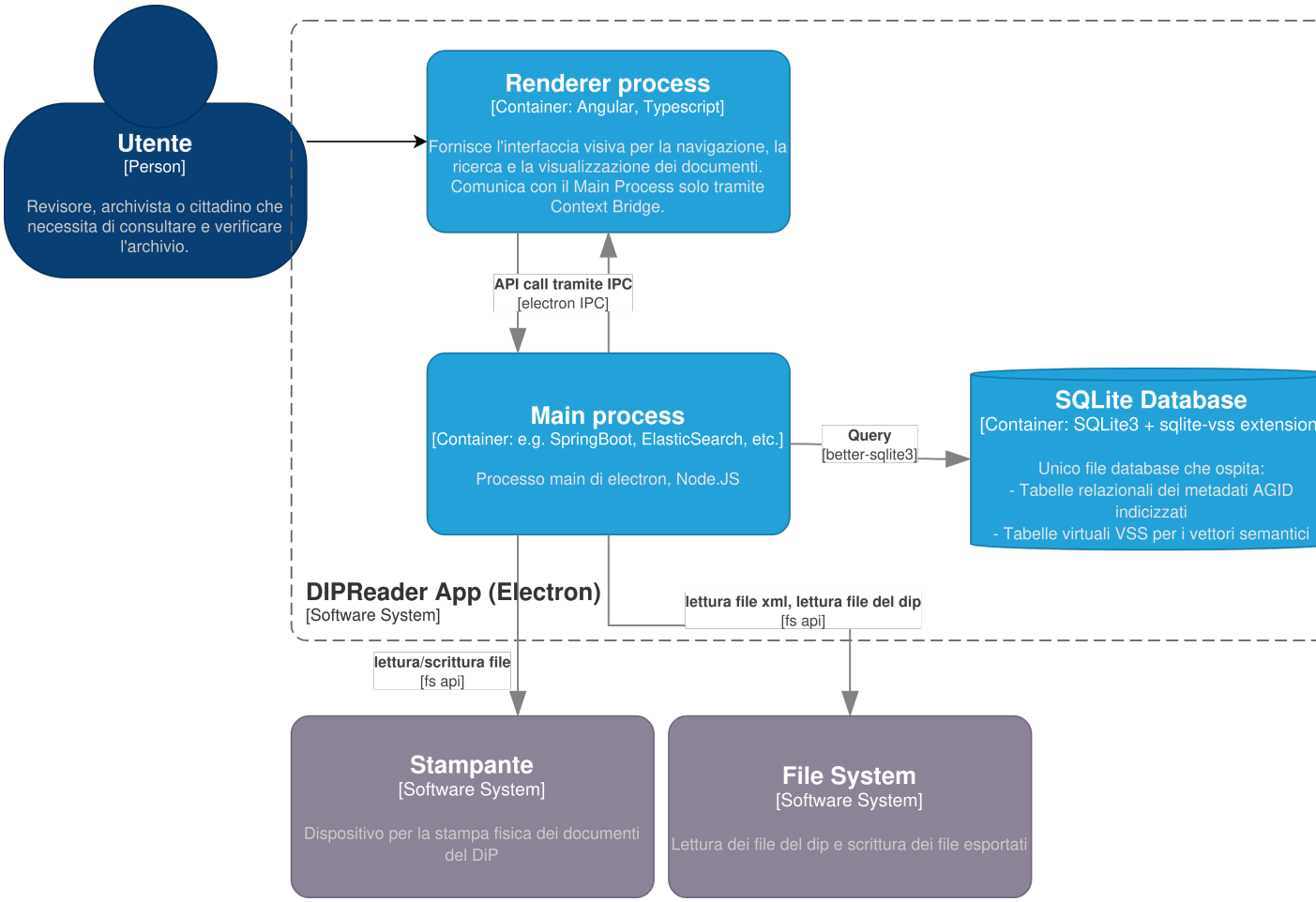

Figura 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione quindi dell'interfaccia è stato scelto di strutturare il container per componenti. Ogni componente a sua volta è strutturato da una serie di services che si occupano di gestire la logica di presentazione, e da una serie di componenti di presentazione che si occupano di mostrare i dati all'utente.
Visto poi che le interazioni tra i diversi componenti di presentazione sono limitate, è stato scelto di strutturare il container in modo che ogni componente sia completamente disaccoppiato dagli altri. Gli unici punti di contatto risultano infatti essere il sistema di routing e i canali IPC, che permettono ai componenti di comunicare tra loro e con il backend.
In presenza poi di componenti come per esempio l'esportazione dei file, che richiedono di essere inseriti in più punti del sistema, è stato scelto di strutturare il container in modo che questi componenti siano facilmente integrabili in qualsiasi punto del sistema, senza dover modificare la struttura del container o dei componenti di presentazione grazie a delle semplici Injection di dipendenze.
È poi stato applicato il pattern Smart and Dumb Components, che permette di separare la logica di presentazione pura dalla logica di business che la coordina.

5.2 [Main]

5.2.1 Tecnologia: Node.js

Descrizione

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Ricerca (FrontEnd)

6.3.1 Descrizione

I componenti di ricerca sono responsabili della gestione delle funzionalità di compilazione dei campi di ricerca.
Come già anticipato il pattern applicato è quello di Smart and Dumb Components, per cui i componenti di ricerca sono suddivisi in componenti di pura presentazione e gestiti da i componenti *"Padri"*, che si occupano di gestire la logica di questi ultimi nonchè le interazioni con altri servizi come la validazione dei filtri, la gestione dei campi di ricerca dinamici e il routing della sezione dei risultati. Nel contesto della ricerca questo coomponente Padre è SearchPageComponent.
Come si può notare questo componente sfrutta la dependency injection nativa di Angular per risolvere le dipendenze. Inoltre sono iniettate le interfaccie dei servizi necessari per il funzionamento e non gli effettivi servizi, applicando il principio di sostituzione di Liskov.

7 Backend - Indexing

7.1 Panoramica

Il modulo di indicizzazione esplora il contenuto del pacchetto DiP e orchestra un flusso dati che va dal *PackageReaderPort* ai repository, garantendo persistenza efficiente delle entità estratte. L'orchestrazione è gestita da *IndexDipUC*, che riceve in modalità asincrona tramite *AsyncGenerator* le entità del pacchetto (DiP, DocumentClass, Process, Document, File) dal *PackageReaderPort*. Ogni entità viene elaborata e persistita nel corrispettivo repository (*dipRepository*, *documentClassRepository*, *processRepository*, *documentRepository*, *fileRepository*). Per i documenti, è previsto un trattamento speciale: il file principale (non gli allegati) subisce un processo di word embedding. L'estrazione del testo e la generazione dei vettori sono orchestrate da *EmbeddingService*, che interagisce con *WordEmbeddingPort* per convertire il contenuto testuale in vettori di embedding. Questi vettori vengono successivamente salvati in *VectorRepository*.
L'indicizzazione avviene in modalità asincrona, garantendo un utilizzo efficiente della memoria: le entità vengono elaborate e salvate man mano che vengono estratte dal pacchetto, senza attendere il completamento dell'esplorazione dell'intero contenuto.
Il modulo espone una inbound port per l'avvio del processo di indicizzazione, attualmente utilizzata da *ApplicationBootstrap* in *main.ts*. Poiché il software è distribuito all'interno del pacchetto DiP stesso, l'indicizzazione viene eseguita automaticamente durante il bootstrap dell'applicazione, senza richiedere intervento manuale dell'utente.

7.2 Componenti

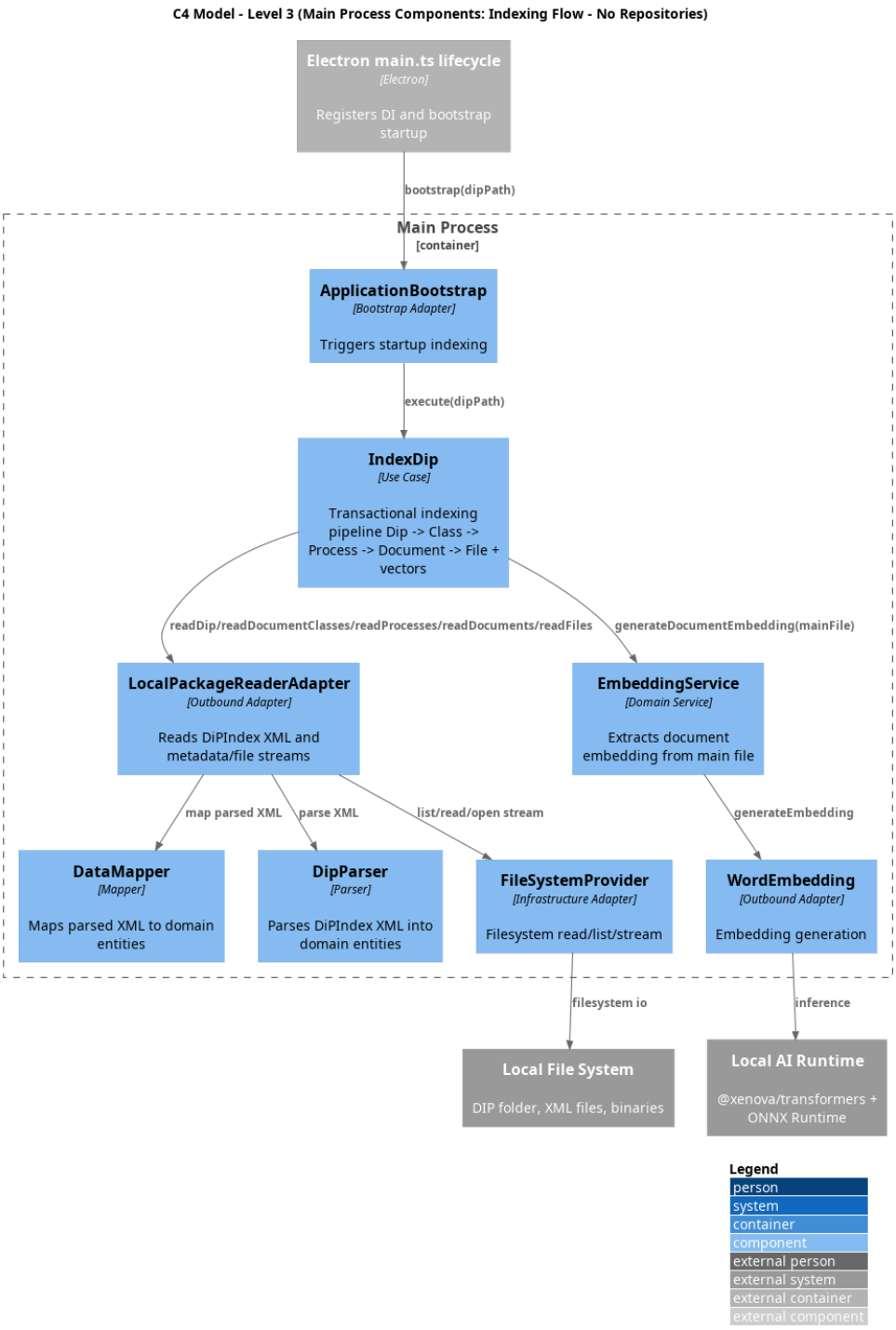

Figura 3: Diagramma C3 - Componenti del modulo di indicizzazione

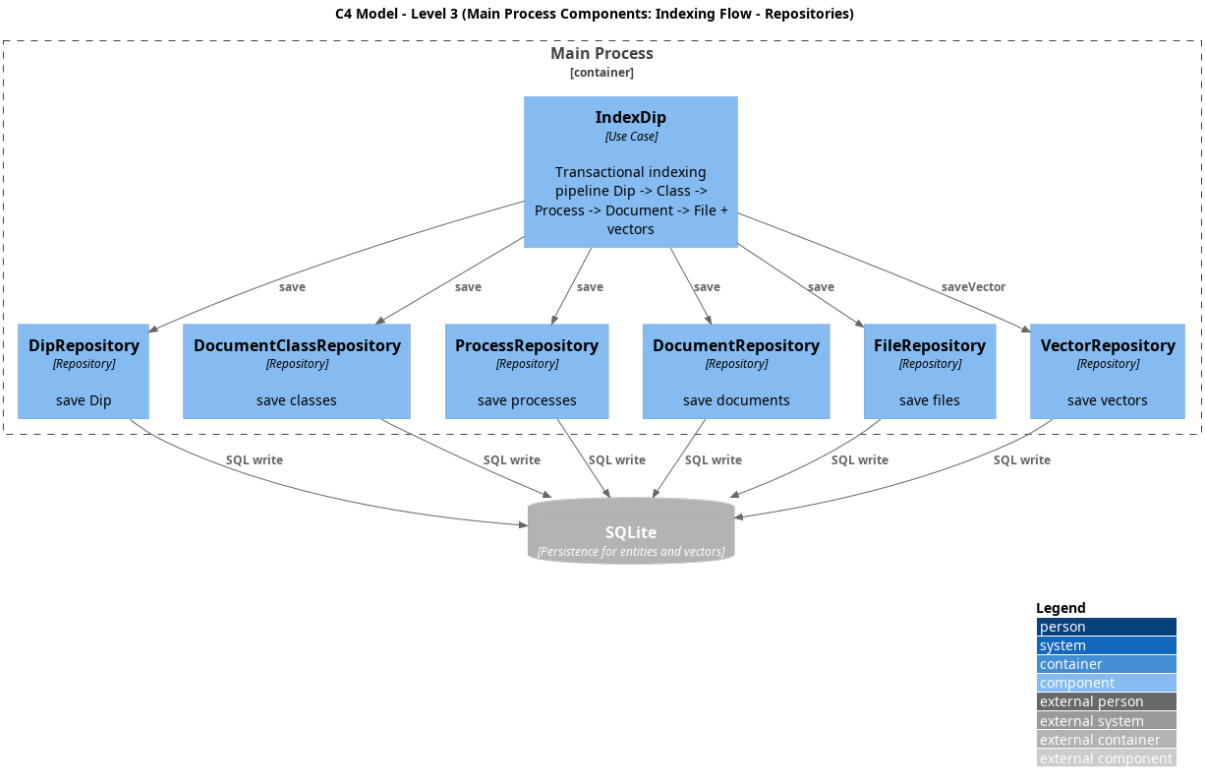

Figura 4: Diagramma C3 - Componenti del modulo di indicizzazione

7.2.1 ApplicationBootstrap, IndexDipUC

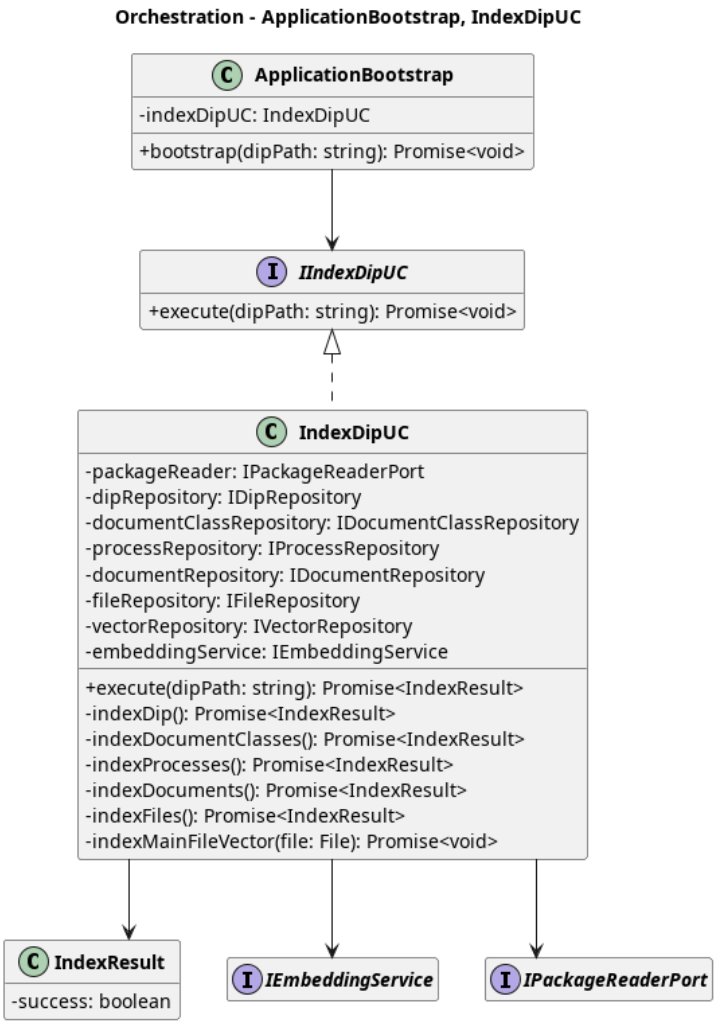

Figura 5: Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione

Prima di tutto è necessario capire la relazione tra le varie entità presenti all'interno di un pacchetto DiP:

- **DiP**: è il pacchetto che viene indicizzato, contiene al suo interno tutte le altre entità.
- **DocumentClass**: rappresenta un aggregato di documenti, ma di fatto nel contesto di un pacchetto DiP rappresenta un aggregato di processi. Essendo il documento gestito all'interno di un processo, è stato aggiunto questo livello di innestamento per rappresentare questa relazione. Un DiP può contenere più DocumentClass.
- **Process**: rappresenta un processo, che contiene al suo interno più documenti.
- **Document**: rappresenta un documento, che contiene al suo interno più file.
- **File**: entità atomica contenuta all'interno di un documento. Può essere il documento principale del file (di cui ne esiste sempre uno e solo uno), oppure un allegato.

Viene spiegato in seguito come queste entità vengono gestite dal modulo di indicizzazione.

ApplicationBootstrap :

- **Descrizione**: adapter responsabile dell'avvio del processo di indicizzazione, utilizzato in *main.ts* per gestire il lifecycle dell'applicazione Electron.
- ***+ bootstrap()**: Promise<void>*: metodo che avvia il processo di indicizzazione, invocando *IndexDipUC.execute()*.

IndexDipUC :

- **Descrizione**: Use Case che orchestra il processo di indicizzazione, coordinando i flussi di dati in arrivo da *PackageReaderPort* verso i repository, effettuando l'embedding dei documenti tramite *WordEmbeddingPort*
- ***+execute(dipPath: string)**: Promise<IndexResult>*: metodo che avvia il processo di indicizzazione, accettando in ingresso il percorso del pacchetto DiP da indicizzare. Restituisce un oggetto *IndexResult* che rappresenta l'esito dell'operazione.
- ***-indexDip()**: Promise<IndexResult>*: Riceve l'entità DiP da *PackageReaderPort.readDip()* e la persiste nel corrispettivo repository *dipRepository*.
- ***-indexDocumentClasses()**: Promise<IndexResult>*: Riceve le DocumentClass da *PackageReaderPort.readDocumentClasses()* tramite AsyncGenerator e le persiste nel repository *documentClassRepository*.
- ***-indexProcesses()**: Promise<IndexResult>*: Riceve i Processi da *PackageReaderPort.readProcesses()* tramite AsyncGenerator e li persiste nel repository *processRepository*.
- ***-indexDocuments()**: Promise<IndexResult>*: Riceve i Documenti da *PackageReaderPort.readDocuments()* tramite AsyncGenerator e li persiste nel repository *documentRepository*. Per ogni documento, identifica il file principale, ne estrae il testo e lo converte in vettore tramite *WordEmbeddingPort*.
- ***-indexFiles()**: Promise<IndexResult>*: Riceve i File da *PackageReaderPort.readFiles()* tramite AsyncGenerator e li persiste nel repository *fileRepository*.
- ***-indexMainFileVector(file: File)**: Promise<IndexResult>*: Estrae il testo del file principale, lo converte in vettore tramite *WordEmbeddingPort* e lo salva in *VectorRepository*.

IndexResult :

- Descrizione: rappresenta l'esito del processo di indicizzazione, contenente una proprietà booleana *success* che indica se l'operazione è stata completata con successo o se si è verificato un errore.
- ***- success: boolean***: indica l'esito del processo di indicizzazione, con *true* se l'operazione è stata completata con successo, o *false* se si è verificato un errore.

7.2.2 EmbeddingService, WordEmbeddingPort

Queste due componenti collaborano per gestire l'estrazione del testo dai file e la generazione dei vettori di embedding, che vengono poi restituiti a IndexDipUC per il salvataggio nella rispettiva *VectorRepository*.

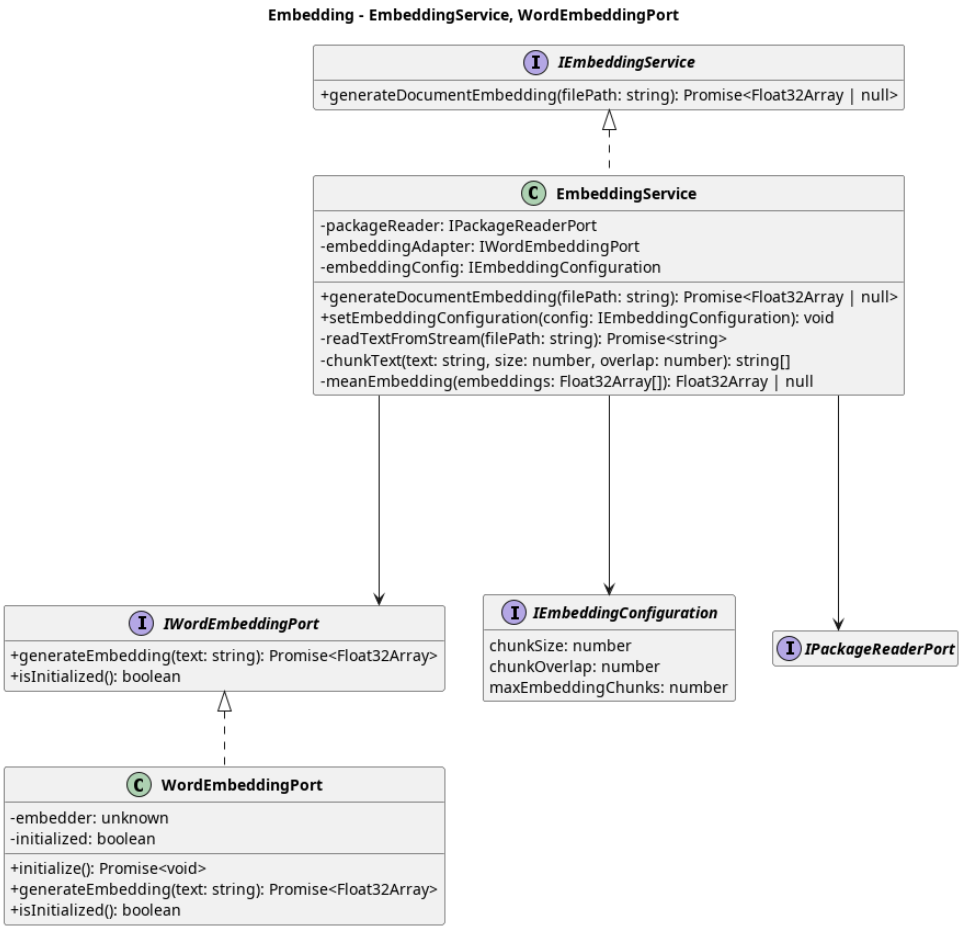

Figura 6: Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione

EmbeddingService :

- **Descrizione**: business service che si occupa di gestire la generazione di vettori a partire dal path del file principale di un documento. Questo servizio non si occupa della persistenza in quanto violerebbe il principio di Single Reponsibility.

- ***+ generateDocumentEmbedding(filePath: string)**: Promise<Float32Array | null>*: tramite *readTextFromStream* legge il contenuto del file principale del documento, che viene successivamente ridotto in chunk tramite *chunkText* e convertito in vettore tramite *WordEmbeddingPort*. Restituisce un array di float a 32 bit che rappresenta il vettore del documento, o null in caso di errore. Questo vettore è costruito come media dei vettori di ogni chunk del testo tramite *meanEmbedding*.
- ***+ setEmbeddingConfiguration(config: IEmbeddingConfiguration)**: void*: permette di aggiornare la configurazione per la generazione degli embedding, accettando in ingresso un oggetto che rispetta l'interfaccia *IEmbeddingConfiguration*.
- ***- readTextFromStream(filePath: string)**: Promise<string>*: legge il contenuto del file principale del documento a partire dal suo path tramite *PackageReaderPort.readFileBytes*, restituendo una stringa di testo.
- ***- chunkText(text: string)**: string[]*: suddivide il testo in chunk di dimensione massima definita da *embeddingConfig.chunkSize*, restituendo un array di stringhe.
- ***- meanEmbedding(embeddings: Float32Array[])**: Float32Array | null*: calcola la media dei vettori di ogni chunk del testo, restituendo un array di float a 32 bit che rappresenta il vettore medio del documento, o null in caso di errore.

IEmbeddingConfiguration :

- **Descrizione**: interfaccia che definisce la configurazione per la generazione degli embedding.
- *chunkSize*: dimensione massima di ogni chunk di testo da convertire in vettore.
- *chunkOverlap*: sovrapposizione in byte tra un chunk e l'altro, per garantire continuità tra i chunk.
- *maxEmbeddingChunks*: numero massimo di chunk da convertire in vettore per ogni documento, permette di mantenere performance stabili con pacchetti contenenti file di dimensioni elevate.

WordEmbeddingPort :

- **Descrizione**: outbound adapter che permette di interagire con ONNX runtime per la generazione di vettori.
- ***+ initialize()**: Promise<void>*: inizializza il modello di embedding (paraphrase-multilingual-MiniLM-L12-v2) tramite la libreria *@xenova/transformers*. Questa operazione è asincrona e deve essere completata prima di poter generare embedding.
- ***+ generateEmbedding(texts: string[])**: Promise<Float32Array[]>*: genera i vettori di embedding per un array di stringhe di testo. Restituisce un array di Float32Array, uno per ogni testo in ingresso.

- ***+ isInitialized(): boolean***: restituisce *true* se il modello è stato inizializzato, *false* altrimenti.

7.2.3 LocalPackageReaderAdapter, DataMapper, DipParser, FileSystemProvider

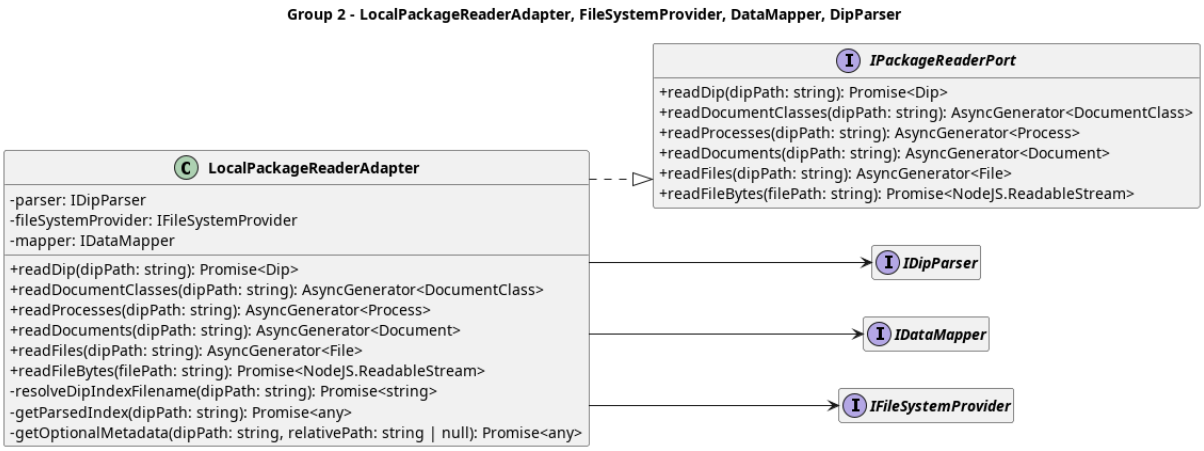

Figura 7: Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione

LocalPackageReaderAdapter :

- **Descrizione**: adapter che implementa *IPackageReaderPort*, responsabile della lettura del pacchetto DiP dal filesystem locale. Orchestr a il flusso di estrazione delle entità coordinando *DipParser*, *FileSystemProvider* e *DataMapper*.

- ***+ readDip(dipPath: string)***: *Promise<Dip>*: legge il file *DiPIndex* dal percorso specificato, lo decodifica tramite *DipParser* e lo trasforma in entità *Dip* tramite *DataMapper*.

- ***+ readDocumentClasses(dipPath: string)***: *AsyncGenerator<DocumentClass>*: genera in modo asincrono le entità *DocumentClass* estratte dal pacchetto. Per ogni classe documentale, legge il file XML corrispondente, lo decodifica e lo trasforma tramite *DataMapper*.

- ***+ readProcesses(dipPath: string)***: *AsyncGenerator<Process>*: genera in modo asincrono le entità *Process* estratte dal pacchetto, leggendo e decodificando i file XML tramite *DipParser* e trasformandoli tramite *DataMapper*.

- ***+ readDocuments(dipPath: string)***: *AsyncGenerator<Document>*: genera in modo asincrono le entità *Document* estratte dal pacchetto, gestendo la lettura dei metadati e la mappatura dei file allegati.

- ***+ readFiles(dipPath: string)***: *AsyncGenerator<File>*: genera in modo asincrono le entità *File* estratte dal pacchetto, distinguendo tra file principale e allegati.

- ***+ readFileBytes(filePath: string)**: Promise<Buffer>*: legge il contenuto binario di un file dal filesystem, utilizzato da *EmbeddingService* per l'estrazione del testo.

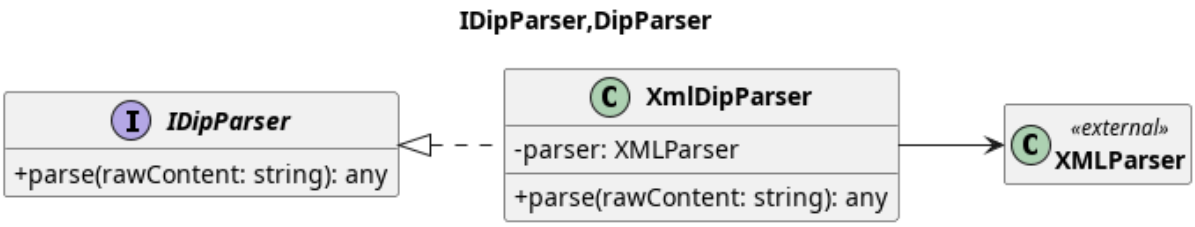

Figura 8: Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione

IDipParser :

- **Descrizione**: interfaccia che permette l'attuazione dello **strategy pattern** per il parsing dei file XML. Il contratto è definito in modo da prevedere la massima flessibilità e facilità di implementazione dei diversi mapper. Viene accettato il contenuto del file (sia esso XML, JSON o altro) e restituito un oggetto JavaScript non tipizzato, che reifica il contenuto del file. L'attuale implementazione prevede il supporto di file XML.
- ***+ parse(rawContent: string)**: any*: effettua il parsing di una stringa XML e restituisce un oggetto JavaScript secondo lo schema fornito.

XmlDipParser :

- **Descrizione**: implementazione concreta di *IDipParser* che utilizza la libreria *fast-xml-parser* per la deserializzazione efficiente e type-safe dei file XML del pacchetto DiP.
- **Configurazione**: le opzioni di parsing sono configurate per gestire correttamente i namespace, gli attributi e le strutture complesse presenti negli schemi XSD forniti.

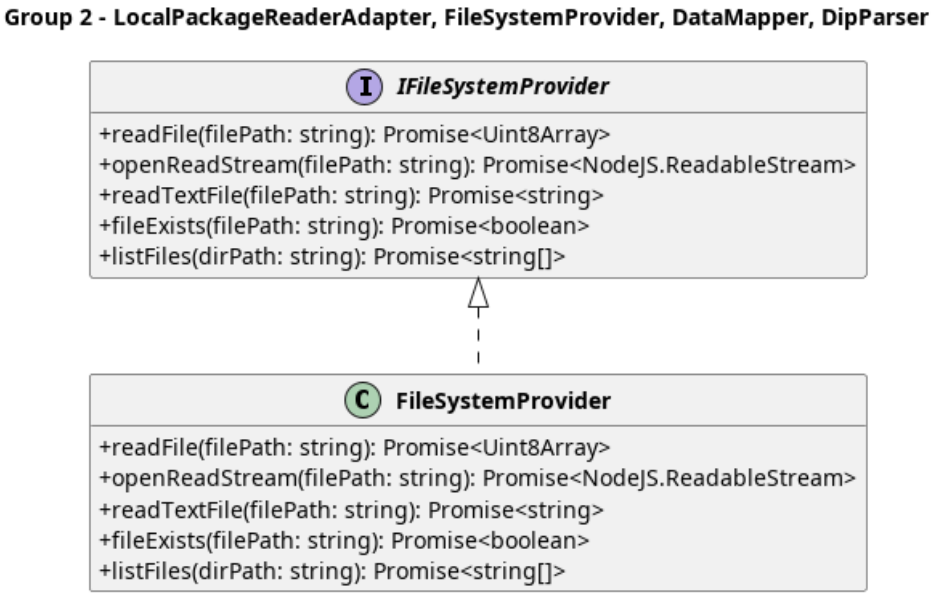

Figura 9: Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione

IFileSystemProvider :

- **Descrizione**: interfaccia che astrae l'accesso al filesystem, permettendo una lettura e una scrittura indipendente dalla piattaforma. Facilita inoltre il testing tramite mock.
- ***+ readFile(filePath: string)**: Promise<string>*: legge il contenuto testuale di un file dal percorso specificato, restituendo una stringa.
- ***+ readFileAsBuffer(filePath: string)**: Promise<Buffer>*: legge il contenuto binario di un file, restituendo un Buffer.
- ***+ listFiles(dirPath: string)**: Promise<string[]>*: elenca i file presenti in una directory, restituendo un array di percorsi relativi.
- ***+ exists(filePath: string)**: Promise<boolean>*: verifica l'esistenza di un file o directory al percorso specificato.

FileSystemProvider :

- **Descrizione**: implementazione concreta di *IFileSystemProvider* che utilizza le API native di Node.js (*fs* e *fs/promises*) per l'accesso al filesystem locale.
- **Caratteristiche**: supporta operazioni asincrone per evitare il blocco del thread principale, essenziale in un contesto Electron dove la responsività dell'UI è critica.

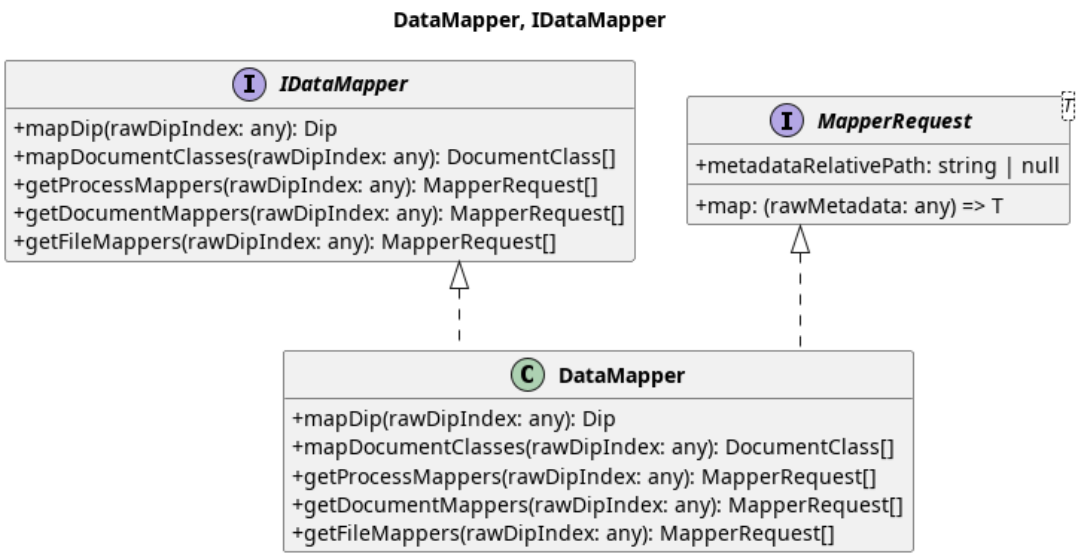

Figura 10: Diagramma C4 - Dettaglio dell'orchestrazione del processo di indicizzazione

IDataMapper :

- **Descrizione**: interfaccia che definisce il contratto per la trasformazione dei dati grezzi estratti dai file XML nei modelli di dominio tipizzati. Utilizza il pattern **MapperRequest** per incapsulare i parametri di mapping e fornire contesto addizionale durante la trasformazione.
- ***+ map(request: MapperRequest<DipIndexXml>)**: Dip*: trasforma i dati grezzi del DiPIndex nelle entità *Dip*. Il *MapperRequest* fornisce il payload XML e il contesto di esecuzione.
- ***+ map(request: MapperRequest<DocumentClassXml>)**: DocumentClass*: trasforma i dati grezzi nelle entità *DocumentClass*, accedendo a metadati contestuali attraverso *MapperRequest*.
- ***+ map(request: MapperRequest<AiPXml>)**: Process*: trasforma i dati grezzi nelle entità *Process*, utilizzando il contesto fornito da *MapperRequest* per correlazioni tra entità.
- ***+ map(request: MapperRequest<DocumentXml>)**: Document*: trasforma i dati grezzi nelle entità *Document*, gestendo i metadati e la relazione con i file attraverso il contesto di *MapperRequest*.
- ***+ map(request: MapperRequest<FileXml>)**: File*: trasforma i dati grezzi nelle entità *File*, distinguendo tra file principale e allegati tramite le informazioni contestuali di *MapperRequest*.

MapperRequest<T> :

- **Descrizione**: interfaccia generica che incapsula i dati da trasformare e fornisce il contesto di esecuzione necessario per il mapping. Permette di passare informazioni addizionali al mapper senza modificare la firma dei metodi.
- ***+ payload: T***: il dato grezzo estratto dal file XML da trasformare nel modello di dominio.
- ***+ context?: Record<string, any>***: oggetto opzionale che contiene metadati contestuali (es. dipId, parentPath, processId) utili durante la trasformazione per stabilire correlazioni tra entità e arricchire il mapping con informazioni di relazione.
- **Vantaggi**: consente di mantenere le firme dei metodi stabili pur permettendo l'espansione del contesto senza breaking changes, facilita il testing tramite iniection del contesto, e rende esplicite le dipendenze contestuali del mapping.

DataMapper :

- **Descrizione**: implementazione concreta di *IDataMapper* che effettua la trasformazione dei dati grezzi nei modelli di dominio, applicando le regole di business specifiche del contesto DiP. Collabora con *DocumentMetadataHashMapper* per la normalizzazione dei metadati.
- **Responsabilità**:
  - Mappatura delle strutture dati XML nelle entità di dominio fortemente tipizzate.
  - Gestione delle relazioni tra entità (es. documento-file, processo-documentclass).
  - Normalizzazione e validazione dei metadati estratti dal pacchetto.
  - Conversione dei tipi primitivi (stringhe, date, etc.) nei corretti tipi TypeScript.

DocumentMetadataHashMapper :

- **Descrizione**: utility class specializzata nella normalizzazione e nella trasformazione dei metadati dei documenti estratti dal pacchetto DiP. Converte la rappresentazione key-value irregolare in strutture coerenti per la persistenza.
- ***+ normalizeMetadata(rawMetadata: Record<string, any>)****: Metadata[]*: normalizza i metadati grezzi, gestendo variabilità nella struttura e nei formati, e restituisce un array di entità *Metadata* fortemente tipizzate.
- **Logica**: implementa regole di trasformazione specifiche per i metadati del contesto archivistico digitale, garantendo coerenza e correttezza dell'informazione.